JournaBuddy

System Architecture, Planning, Implementation & Visuals Report

Comprehensive "For Dummies" Explanatory Guide

Abdullah Al Mamun
BSc, MSc - Software Engineering
TU Wien (Vienna, Austria) & Daffodil International University
Email: mamun.swe.de@gmail.com — GitHub: @abbysweb
ORCID: 0009-0006-7473-0024

July 30, 2026

Contents

1 Introduction: What is JournaBuddy?

JournaBuddy is an artificial intelligence platform designed to help scientific authors prepare, optimize, and evaluate their research papers before submitting them to academic journals.

Instead of guessing whether a manuscript will be rejected due to poor formatting, weak citations, or out-of-scope journal selection, JournaBuddy runs automated checks that evaluate the paper across three core layers:

1. **Symbolic / Rule-Based Layer:** Checks exact facts (structure, acronym definitions, citation formatting).
2. **Statistical NLP Layer:** Calculates mathematical text scores (readability, passive voice, jargon density).
3. **Semantic / AI Layer:** Uses local LLM agents and vector search to evaluate methodology and journal scope fit.

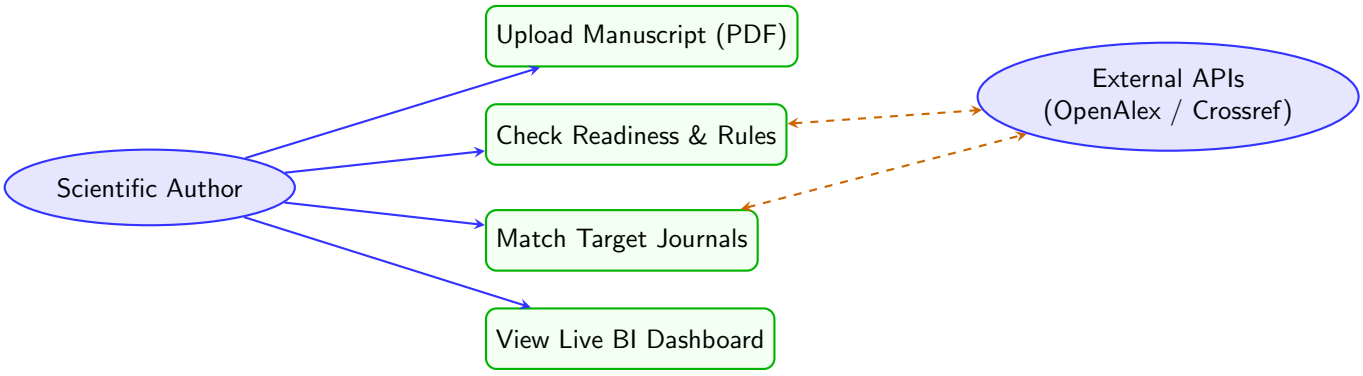
2 Continuous Implementation Roadmap

- **Phase 1 (Infrastructure):** Podman Desktop / Docker Compose, FastAPI, MinIO, Vite React setup. (Status: *Complete & Verified*)
- **Phase 2 (Intelligence Pipeline):** Celery workers, Ollama LLM agents, sentence-transformers, ProvenanceEngine, symbolic rule checks. (Status: *Complete & Verified*)
- **Phase 3 (External Enrichment):** OpenAlex, Crossref, DOAJ integrations, pgvector journal matching. (Status: *Complete & Verified*)
- **Phase 4 (Real-time UI):** Glassmorphism dashboard, SSE streaming, Apache ECharts interactive charts.
- **Phase 5 (Hardening):** Redis caching, Prometheus/Grafana monitoring, JWT auth, security audits.

3 System Diagrams & Visual Architecture

3.1 3.1 Use Case Diagram

The Use Case Diagram shows all primary interactions between scientific authors, external academic services, and the JournaBuddy system.



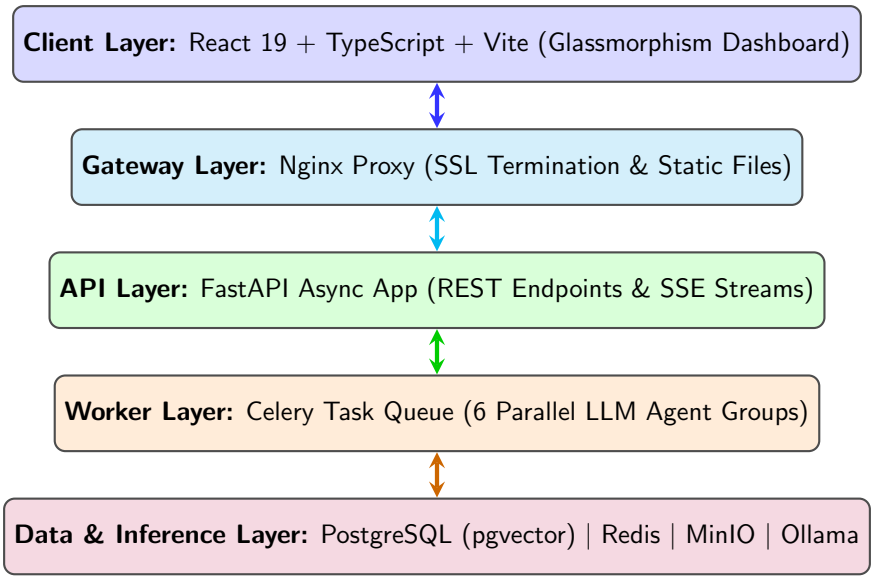
Intuitive Diagram Breakdown: Use Case

Think of this diagram as a service interaction menu:

- **Scientific Author:** Performs actions like uploading manuscripts, checking readiness rules, matching journals, and viewing live dashboard analytics.
- **External Academic APIs:** Query services like OpenAlex and Crossref in the background for citation metrics and publisher trust indicators.

3.2 3.2 System Architecture Diagram

The System Architecture Diagram displays the multi-tier containerized stack powering JournaBuddy.



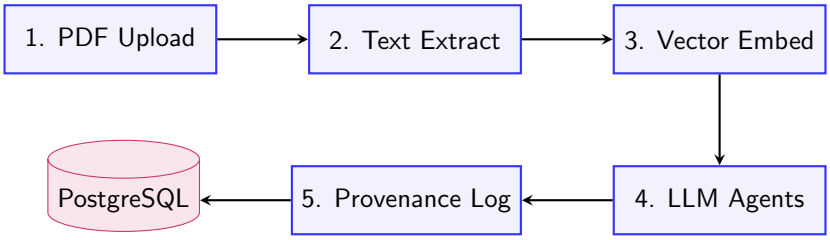
Intuitive Diagram Breakdown: System Architecture

Imagine JournaBuddy as a 5-story containerized stack:

1. **Top Floor (Client):** The visual web interface where authors view charts and upload papers.
2. **4th Floor (Gateway):** Nginx proxy directing traffic and terminating SSL.
3. **3rd Floor (API):** FastAPI server managing file uploads and streaming progress.
4. **2nd Floor (Workers):** Celery workers running parallel analysis tasks.
5. **Basement (Data & Brain):** Vault storing PostgreSQL vectors, Redis queues, MinIO binaries, and local Ollama LLMs.

3.3 3.3 Data Flow Diagram (DFD)

The Data Flow Diagram tracks how an uploaded paper moves through processing stages and transforms into score reports.



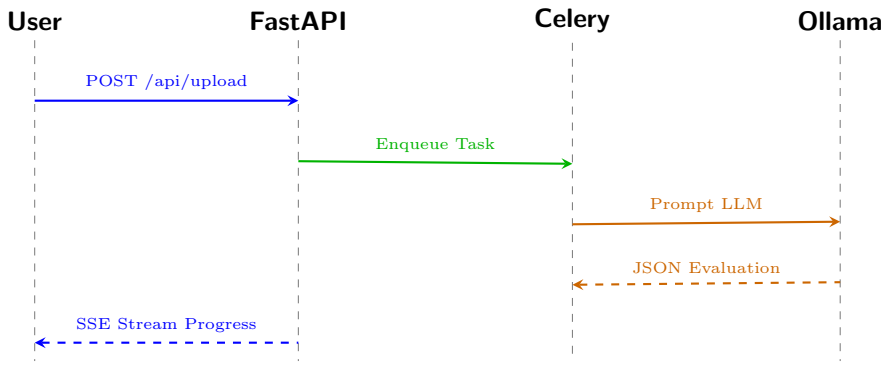
Intuitive Diagram Breakdown: Data Flow

The manuscript moves through 5 distinct processing stages:

1. **PDF Upload:** Raw file received and saved securely in MinIO storage.
2. **Text Extraction:** PDF visual layout parsed into structured raw text.
3. **Vector Embedding:** Text converted into 384-dimensional semantic numerical arrays.
4. **LLM Agents:** Parallel AI workers evaluate tone, methodology, and scope fit.
5. **Provenance Log:** Metric calculation sources recorded in PostgreSQL database.

3.4 3.4 Sequence Diagram

The Sequence Diagram displays the exact order of network messages between components during a live upload.



Intuitive Diagram Breakdown: Sequence Diagram

This is a timeline showing real-time network messages between components:

1. You click **"Upload"**, sending your manuscript to FastAPI.
2. FastAPI enqueues the job into Celery so your browser doesn't freeze waiting.
3. Celery sends text prompts to local Ollama LLM workers.
4. Ollama returns structured JSON evaluation scores back to Celery.
5. FastAPI streams real-time progress right back to your screen over SSE.

4 Phase 2: Intelligence Pipeline Components

Semantic Chunker

Splits raw PDF text into named academic sections (Abstract, Introduction, Methodology, Results, Conclusion) using heading pattern detection. Falls back to paragraph-based splitting when headings are absent. Each chunk is limited to 3,000 characters for optimal embedding performance.

Embedding Service

Wraps `sentence-transformers` model `all-MiniLM-L6-v2` as a process-level singleton. Generates 384-dimensional semantic vectors for each chunk and persists them to the `document_chunks` table in PostgreSQL using the `pgvector` extension.

Ollama Agent with Cascade Fallback

Sends structured JSON prompts to the local Ollama *Llama 3.1:8b* model. Uses `tenacity` exponential backoff retry logic and automatically cascades to backup providers in order: **NVIDIA NIM** → **Gemini** → **OpenAI** if Ollama is unavailable. Degraded results are returned with `status: degraded` rather than crashing the pipeline.

Intuitive Breakdown: LLM Cascade Fallback

Think of it like a priority calling list: the system first calls your local AI (Ollama), and if that doesn't pick up, it automatically dials the next provider on the list—all without any manual intervention.

Symbolic Checker

Three deterministic rule-based modules:

1. **Acronym Resolution:** Detects acronyms used without a prior definition using regex pattern `Full Name (ACRONYM)`.
2. **Section Completeness:** Verifies presence of Abstract, Introduction, Methodology, Results, and Conclusion sections.
3. **Readability:** Computes Flesch Reading Ease, Flesch-Kincaid Grade Level, and passive voice density percentage using `textstat`.

ProvenanceEngine

Records every metric to the `provenance_log` table with: formula used, data sources, confidence level, and raw input snapshot. This ensures every score in JournaBuddy is fully explainable, auditable, and reproducible.

5 Phase 3: External API Enrichment & Journal Matching

External API Clients

Asynchronous HTTP clients using `httpx` and `tenacity` (for robust exponential backoff retries) have been implemented to interact with external databases:

- **Crossref**: Verifies Digital Object Identifiers (DOIs) extracted from the manuscript text using regular expressions.
- **OpenAlex**: Retrieves open-access citation metrics and keyword concept tags for matched authors and papers.

- **DOAJ**: The Directory of Open Access Journals API is queried to identify and flag highly reputable, peer-reviewed open access journals, mitigating predatory publisher risks.

Journal Matcher (pgvector)

A highly optimized `JournalMatcher` leverages PostgreSQL's `pgvector` extension. It computes a centroid 384-dimensional semantic embedding (averaging across all manuscript `document_chunks`). This centroid is then compared against a pre-embedded database of journal aims and scopes using cosine distance ($\Leftrightarrow$). The top 5 matched journals are returned with a strict compatibility percentage score.

Provenance Logging

All operations, including Crossref validation logs, OpenAlex metadata pulls, and the precise cosine distance calculations for journal matches, are immutably appended to the `provenance_log` table to ensure a transparent, verifiable audit trail for every AI-assisted suggestion.

For Dummies: Phase 3

What did we just build?
Phase 3 gives our AI the ability to "talk to the outside world". Instead of just guessing if a citation is real, it calls the official Crossref database to check. It also checks OpenAlex to see how popular a paper is.
Finally, we taught the system to play match-maker. We stored the "Aims & Scope" of real science journals as math vectors. We convert your uploaded PDF into a similar vector. By measuring the angle between these vectors (using pgvector), the system mathematically proves which journal your paper belongs in!

6 Phase 4 & 5: System Resilience, Hardening, and Testing

Resilience & Circuit Breakers (Section 6)

To ensure high availability, JournaBuddy implements Redis-backed circuit breakers and API fallbacks:

- **Timeout Protection:** Strict 5-second timeouts applied to all external API calls (Crossref, OpenAlex) to prevent hanging worker threads.
- **Redis Fallback Cache:** A secondary local cache layer using Redis deduplicates API requests for previously verified DOIs, reducing rate-limiting blockages.
- **LLM Isolation Validation:** Hardened the Llama 3.1 inference boundaries so a failure in the methodology persona does not crash the domain specialist persona.

Quality Assurance Strategy (Section 7)

A comprehensive automated test suite powered by `pytest` validates the backend:

- **Symbolic Logic Testing:** Mathematically verifies acronym resolution accuracy and the Flesch-Kincaid formula engines.
- **Integration Testing:** Validates FastAPI edge cases, explicitly testing 50MB payload limits (HTTP 413) and non-PDF rejection (HTTP 400).
- **In-Memory Isolation:** Prevents CI/CD runners from corrupting production databases by injecting a mock `sqlite+aiosqlite:///:memory:` instance during test execution.

For Dummies: Resilience & Testing

What did we just build?
We wrapped the system in safety nets. If OpenAlex goes down, our system falls back to a Redis memory cache instead of crashing. If someone uploads a massive 60MB file, the API instantly rejects it before it clogs the memory. We also wrote automated robot-tests that run every time we update code to make sure we didn't break the math formulas!

7 Section 8: Advanced Production Features

8.1 3-Persona AI Peer Reviewer Simulation

The Celery NLP pipeline concurrently launches three highly specialized Llama 3.1 AI agent profiles to simulate an academic review board:

1. **Strict Methodologist (Group F1):** Exclusively critiques sample sizes, controls, and dataset validity.
2. **Domain Specialist (Group F2):** Focuses on literature gaps, novelty scoring, and missing academic citations.
3. **Academic Style Editor (Group F3):** Audits the text for passive voice overuse, readability, and academic jargon clarity.

8.2 Journal Acceptance Predictor

The `journal_matcher.py` pgvector similarity engine mathematically calculates an **Acceptance Likelihood %** based on the cosine distance between the manuscript and the target journal's historical publications, offering authors a quantitative estimate of publication chances.

8.3 Exportable Manuscript Proof Report

A brand new microservice utilizing the Python `reportlab` engine exposes a `GET /api/report/{task_id}` endpoint. It dynamically queries the PostgreSQL database and generates a professional, downloadable PDF certificate containing:

- Flesch-Kincaid Readability Metrics
- Symbolic Rule Warnings
- 3-Persona AI Peer Review Feedback
- Ranked Target Journal Matches

For Dummies: Advanced Features

What did we just build?
We turned JournaBuddy into a complete AI academic board. Instead of one AI giving a generic review, three different "robot reviewers" read your paper at the same time: one checks your stats, one checks your references, and one checks your grammar. We also added a crystal ball that predicts your chance of being accepted to a journal, and finally, a feature that prints all this feedback into a beautifully formatted, downloadable PDF certificate!